

Here f_E is the function value at an “extrapolated” point somewhat further along the proposed new direction. Also define Δf to be the magnitude of the largest decrease along one particular direction of the present basic procedure iteration. (Δf is a positive number.) Then:

1. If $f_E \geq f_0$, then keep the old set of directions for the next basic procedure, because the average direction $\mathbf{P}_N - \mathbf{P}_0$ is all played out.
2. If $2(f_0 - 2f_N + f_E)[(f_0 - f_N) - \Delta f]^2 \geq (f_0 - f_E)^2 \Delta f$, then keep the old set of directions for the next basic procedure, because either (i) the decrease along the average direction was not primarily due to any single direction's decrease, or (ii) there is a substantial second derivative along the average direction and we seem to be near to the bottom of its minimum.

The following routine implements Powell's method in the version just described. In the routine, `xi` is the matrix whose columns are the set of directions $\mathbf{n}_i$; otherwise the correspondence of notation should be self-evident. If the function to be minimized is provided as a functor `Func`

```
struct Func {
    Doub operator()(VecDoub_I &x);
};
```

then the normal calling sequence for `Powell` looks something like this:

```
VecDoub p = ...;
Func func;
Powell<Func> powell(func);
p=powell.minimize(p);
```

OK to overwrite initial guess.

The function value at the minimum is available as `powell.fret`.

If, on the other hand, the function to be minimized is provided as a normal C++ function,

```
Doub func(VecDoub_I &x);
```

then the constructor call looks like this instead:

```
Powell<Doub (VecDoub_I &)> powell(func);
```

Note that the constructor takes an optional argument that specifies the function tolerance for the minimization.

```
template <class T>
struct Powell : Linemethod<T> {
    Multidimensional minimization by Powell's method.
```

`mins_ndim.h`

```
    Int iter;
    Doub fret;
    using Linemethod<T>::func;
    using Linemethod<T>::linmin;
    using Linemethod<T>::p;
    using Linemethod<T>::xi;
    const Doub ftol;
    Powell(T &func, const Doub ftoll=3.0e-8) : Linemethod<T>(func),
        ftol(ftoll) {}
```

Value of the function at the minimum.
Variables from a templated base class are not automatically inherited.

Constructor arguments are `func`, the function or functor to be minimized, and an optional argument `ftoll`, the fractional tolerance in the function value such that failure to decrease by more than this amount on one iteration signals doneness.

```
VecDoub minimize(VecDoub_I &pp)
```

Minimization of a function or functor `n` variables. Input consists of an initial starting point `pp[0..n-1]`. The initial matrix `ximat[0..n-1][0..n-1]`, whose columns contain the initial set of directions, is set to the `n` unit vectors. Returned is the best point found, at which point `fret` is the minimum function value and `iter` is the number of iterations taken.

```

{
    Int n=pp.size();
    MatDoub ximat(n,n,0.0);
    for (Int i=0;i<n;i++) ximat[i][i]=1.0;
    return minimize(pp,ximat);
}
VecDoub minimize(VecDoub_I &pp, MatDoub_IO &ximat)
Alternative interface: Input consists of the initial starting point pp[0..n-1] and an initial
matrix ximat[0..n-1][0..n-1], whose columns contain the initial set of directions. On
output ximat is the then-current direction set.
{
    const Int ITMAX=200;           Maximum allowed iterations.
    const Doub TINY=1.0e-25;      A small number.
    Doub fptt;
    Int n=pp.size();
    p=pp;
    VecDoub pt(n),ptt(n);
    xi.resize(n);
    fret=func(p);
    for (Int j=0;j<n;j++) pt[j]=p[j]; Save the initial point.
    for (iter=0;;++iter) {
        Doub fp=fret;
        Int ibig=0;
        Doub del=0.0;              Will be the biggest function decrease.
        for (Int i=0;i<n;i++) { In each iteration, loop over all directions in the set.
            for (Int j=0;j<n;j++) xi[j]=ximat[j][i]; Copy the direction,
            fptt=fret;
            fret=linmin();          minimize along it,
            if (fptt-fret > del) {   and record it if it is the largest decrease
                del=fptt-fret;      so far.
                ibig=i+1;
            }
        }
        Here comes the termination criterion:
        if (2.0*(fp-fret) <= ftol*(abs(fp)+abs(fret))+TINY) {
            return p;
        }
        if (iter == ITMAX) throw("powell exceeding maximum iterations.");
        for (Int j=0;j<n;j++) { Construct the extrapolated point and the
            ptt[j]=2.0*p[j]-pt[j]; average direction moved. Save the
            xi[j]=p[j]-pt[j]; old starting point.
            pt[j]=p[j];
        }
        fptt=func(ptt);          Function value at extrapolated point.
        if (fptt < fp) {
            Doub t=2.0*(fp-2.0*fret+fptt)*SQR(fp-fret-del)-del*SQR(fp-fptt);
            if (t < 0.0) {
                fret=linmin();      Move to the minimum of the new direc-
                for (Int j=0;j<n;j++) { tion, and save the new direction.
                    ximat[j][ibig-1]=ximat[j][n-1];
                    ximat[j][n-1]=xi[j];
                }
            }
        }
    }
}
};

```

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapter 7.[1]

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 464–467.[2]
- Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), pp. 259–262.

10.8 Conjugate Gradient Methods in Multidimensions

Consider now the case where you are able to calculate, at a given N -dimensional point $\mathbf{P}$, not just the value of a function $f(\mathbf{P})$ but also the gradient (vector of first partial derivatives) $\nabla f(\mathbf{P})$.

A rough counting argument will show how advantageous it is to use the gradient information: Suppose that the function f is roughly approximated as a quadratic form, as above in equation (10.7.1),

$$f(\mathbf{x}) \approx c - \mathbf{b} \cdot \mathbf{x} + \frac{1}{2} \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} \quad (10.8.1)$$

Then the number of unknown parameters in f is equal to the number of free parameters in $\mathbf{A}$ and $\mathbf{b}$, which is $\frac{1}{2}N(N+1)$, which we see to be of order N^2 . Changing any one of these parameters can move the location of the minimum. Therefore, we should not expect to be able to *find* the minimum until we have collected an equivalent information content, of order N^2 numbers.

In the direction set methods of §10.7, we collected the necessary information by making on the order of N^2 separate line minimizations, each requiring “a few” (but sometimes a *big* few!) function evaluations. Now, each evaluation of the gradient will bring us N new components of information. If we use them wisely, we should need to make only of order N separate line minimizations. That is in fact the case for the algorithms in this section and the next.

A factor of N improvement in computational speed is not necessarily implied. As a rough estimate, we might imagine that the calculation of *each component* of the gradient takes about as long as evaluating the function itself. In that case there will be of order N^2 equivalent function evaluations both with and without gradient information. Even if the advantage is not of order N , however, it is nevertheless quite substantial: (i) Each calculated component of the gradient will typically save not just one function evaluation, but a number of them, equivalent to, say, a whole line minimization. (ii) There is often a high degree of redundancy in the formulas for the various components of a function’s gradient. When this is so, especially when there is also redundancy with the calculation of the function, the calculation of the gradient may cost significantly less than N function evaluations.

A common beginner’s error is to assume that any reasonable way of incorporating gradient information should be about as good as any other. This line of thought leads to the following *not-very-good* algorithm, the *steepest descent method*:

Steepest Descent: Start at a point $\mathbf{P}_0$. As many times as needed, move from point $\mathbf{P}_i$ to the point $\mathbf{P}_{i+1}$ by minimizing along the line from $\mathbf{P}_i$ in the direction of the local downhill gradient $-\nabla f(\mathbf{P}_i)$.

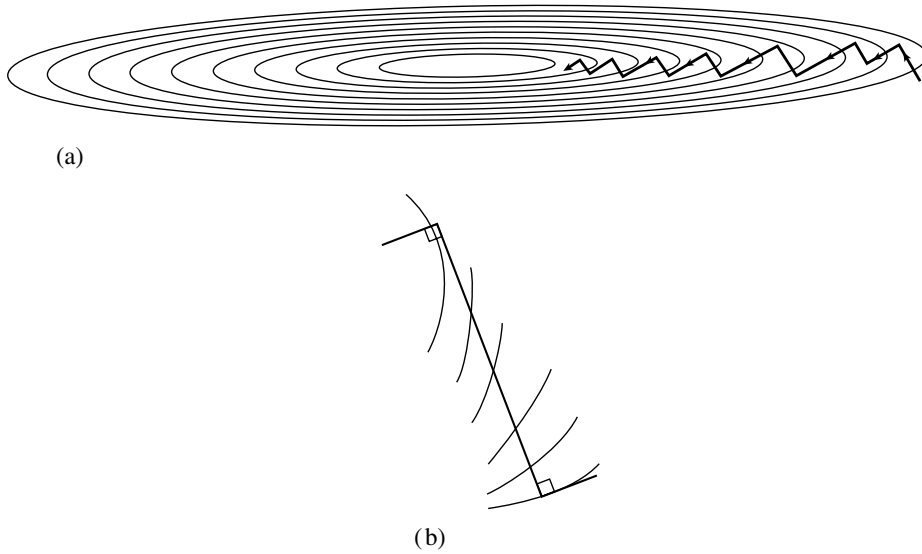


Figure 10.8.1. (a) Steepest descent method in a long, narrow “valley.” While more efficient than the strategy of Figure 10.7.1, steepest descent is nonetheless an inefficient strategy, taking many steps to reach the valley floor. (b) Magnified view of one step: A step starts off in the local gradient direction, perpendicular to the contour lines, and traverses a straight line until a local minimum is reached, where the traverse is parallel to the local contour lines.

The problem with the steepest descent method (which, incidentally, goes back to Cauchy), is similar to the problem that was shown in Figure 10.7.1. The method will perform many small steps in going down a long, narrow valley, even if the valley is a perfect quadratic form. You might have hoped that, say in two dimensions, your first step would take you to the valley floor, the second step directly down the long axis; but remember that the new gradient at the minimum point of any line minimization is perpendicular to the direction just traversed. Therefore, with the steepest descent method, you *must* make a right angle turn, which does *not*, in general, take you to the minimum. (See Figure 10.8.1.)

Just as in the discussion that led up to equation (10.7.5), we really want a way of proceeding not down the new gradient, but rather in a direction that is somehow constructed to be *conjugate* to the old gradient, and, insofar as possible, to all previous directions traversed. Methods that accomplish this construction are called *conjugate gradient* methods.

In §2.7 we discussed the conjugate gradient method as a technique for solving linear algebraic equations by minimizing a quadratic form. That formalism can also be applied to the problem of minimizing a function *approximated* by the quadratic form (10.8.1). Recall that, starting with an arbitrary initial vector $\mathbf{g}_0$ and letting $\mathbf{h}_0 = \mathbf{g}_0$, the conjugate gradient method constructs two sequences of vectors from the recurrence

$$\mathbf{g}_{i+1} = \mathbf{g}_i - \lambda_i \mathbf{A} \cdot \mathbf{h}_i \quad \mathbf{h}_{i+1} = \mathbf{g}_{i+1} + \gamma_i \mathbf{h}_i \quad i = 0, 1, 2, \dots \quad (10.8.2)$$

The vectors satisfy the orthogonality and conjugacy conditions

$$\mathbf{g}_i \cdot \mathbf{g}_j = 0 \quad \mathbf{h}_i \cdot \mathbf{A} \cdot \mathbf{h}_j = 0 \quad \mathbf{g}_i \cdot \mathbf{h}_j = 0 \quad j < i \quad (10.8.3)$$

The scalars λ_i and γ_i are given by

$$\lambda_i = \frac{\mathbf{g}_i \cdot \mathbf{g}_i}{\mathbf{h}_i \cdot \mathbf{A} \cdot \mathbf{h}_i} = \frac{\mathbf{g}_i \cdot \mathbf{h}_i}{\mathbf{h}_i \cdot \mathbf{A} \cdot \mathbf{h}_i} \quad (10.8.4)$$

$$\gamma_i = \frac{\mathbf{g}_{i+1} \cdot \mathbf{g}_{i+1}}{\mathbf{g}_i \cdot \mathbf{g}_i} \quad (10.8.5)$$

Equations (10.8.2) – (10.8.5) are simply equations (2.7.32) – (2.7.35) for a symmetric $\mathbf{A}$ in a new notation. (A self-contained derivation of these results in the context of function minimization is given by Polak [1].)

Now suppose that we knew the Hessian matrix $\mathbf{A}$ in equation (10.8.1). Then we could use the construction (10.8.2) to find successively conjugate directions $\mathbf{h}_i$ along which to line-minimize. After N such, we would efficiently have arrived at the minimum of the quadratic form. But we don't know $\mathbf{A}$.

Here is a remarkable theorem to save the day: Suppose we happen to have $\mathbf{g}_i = -\nabla f(\mathbf{P}_i)$, for some point $\mathbf{P}_i$, where f is of the form (10.8.1). Suppose also that we proceed from $\mathbf{P}_i$ along the direction $\mathbf{h}_i$ to the local minimum of f located at some point $\mathbf{P}_{i+1}$ and then set $\mathbf{g}_{i+1} = -\nabla f(\mathbf{P}_{i+1})$. Then, this $\mathbf{g}_{i+1}$ is the same vector as would have been constructed by equation (10.8.2). (And we have constructed it without knowledge of $\mathbf{A}$!)

Proof: By equation (10.7.3), $\mathbf{g}_i = -\mathbf{A} \cdot \mathbf{P}_i + \mathbf{b}$ and

$$\mathbf{g}_{i+1} = -\mathbf{A} \cdot (\mathbf{P}_i + \lambda \mathbf{h}_i) + \mathbf{b} = \mathbf{g}_i - \lambda \mathbf{A} \cdot \mathbf{h}_i \quad (10.8.6)$$

with λ chosen to take us to the line minimum. But at the line minimum $\mathbf{h}_i \cdot \nabla f = -\mathbf{h}_i \cdot \mathbf{g}_{i+1} = 0$. This latter condition is easily combined with (10.8.6) to solve for λ . The result is exactly the expression (10.8.4). But with this value of λ , (10.8.6) is the same as (10.8.2), q.e.d.

We have, then, the basis of an algorithm that requires neither knowledge of the Hessian matrix $\mathbf{A}$ nor even the storage necessary to store such a matrix. A sequence of directions $\mathbf{h}_i$ is constructed, using only line minimizations, evaluations of the gradient vector, and an auxiliary vector to store the latest in the sequence of $\mathbf{g}$'s.

The algorithm described so far is the original Fletcher-Reeves version of the conjugate gradient algorithm. Later, Polak and Ribiere introduced one tiny, but sometimes significant, change. They proposed using the form

$$\gamma_i = \frac{(\mathbf{g}_{i+1} - \mathbf{g}_i) \cdot \mathbf{g}_{i+1}}{\mathbf{g}_i \cdot \mathbf{g}_i} \quad (10.8.7)$$

instead of equation (10.8.5). “Wait,” you say, “aren't they equal by the orthogonality conditions (10.8.3)?” They are equal for exact quadratic forms. In the real world, however, your function is not exactly a quadratic form. Arriving at the supposed minimum of the quadratic form, you may still need to proceed for another set of iterations. There is some evidence [2] that the Polak-Ribiere formula accomplishes the transition to further iterations more gracefully: When it runs out of steam, it tends to reset $\mathbf{h}$ to be down the local gradient, which is equivalent to beginning the conjugate gradient procedure anew.

The following routine implements the Polak-Ribiere variant, which we recommend; but changing one program line, as shown, will give you Fletcher-Reeves. The

routine presumes the existence of a functor (not a function) that returns the function value by overloading operator() and also provides a function to set the vector gradient $df[0..n-1]$ evaluated at the input point p . Here's an example for the function $x_0^2 + x_1^2$:

```
struct Funcd {
    Doub operator() (VecDoub_I &x)
    {
        return x[0]*x[0]+x[1]*x[1];
    }
    void df(VecDoub_I &x, VecDoub_O &deriv)
    {
        deriv[0]=2.0*x[0];
        deriv[1]=2.0*x[1];
    }
};
```

Name Funcd is arbitrary.

Name df is fixed.

To use `frprmn`, you need statements like the following:

```
Funcd funcd;
Frprmn<Funcd> frprmn(funcd);
VecDoub p = ...;
p=frprmn.minimize(p);
```

OK to overwrite initial guess.

The function value at the minimum is available as `frprmn.fret`. Note that the constructor takes an optional argument that specifies the function tolerance for the minimization.

The routine calls `linmin` to do the line minimizations. As already discussed, you may wish to use a modified version of `linmin` that uses `Dbrent` instead of `Brent`, i.e., that uses the gradient in doing the line minimizations. See note below (§10.8.1).

`mins_ndim.h`

```
template <class T>
struct Frprmn : Linemethod<T> {
    Multidimensional minimization by the Fletcher-Reeves-Polak-Ribiere method.
    Int iter;
    Doub fret;
    using Linemethod<T>::func;
    using Linemethod<T>::linmin;
    using Linemethod<T>::p;
    using Linemethod<T>::xi;
    const Doub ftol;
    Frprmn(T &funcd, const Doub ftoll=3.0e-8) : Linemethod<T>(funcd),
        ftol(ftoll) {}
    Value of the function at the minimum.
    Variables from a templated base class
    are not automatically inherited.
```

Constructor arguments are `funcd`, the function or functor to be minimized, and an optional argument `ftoll`, the fractional tolerance in the function value such that failure to decrease by more than this amount on one iteration signals doneness.

`VecDoub minimize(VecDoub_I &pp)`

Given a starting point `pp[0..n-1]`, performs the minimization on a function whose value and gradient are provided by a functor `funcd` (see text).

```
{
    const Int ITMAX=200;
    const Doub EPS=1.0e-18;
    const Doub GTOL=1.0e-8;
    Here ITMAX is the maximum allowed number of iterations; EPS is a small number to
    rectify the special case of converging to exactly zero function value; and GTOL is the
    convergence criterion for the zero gradient test.
    Doub gg,dgg;
    Int n=pp.size();
    p=pp;
    VecDoub g(n),h(n);
    Initializations.
```

```

xi.resize(n);
Doub fp=func(p);
func.df(p,xi);
for (Int j=0;j<n;j++) {
    g[j] = -xi[j];
    xi[j]=h[j]=g[j];
}
for (Int its=0;its<ITMAX;its++) {      Loop over iterations.
    iter=its;
    fret=linmin();                     Next statement is one possible return:
    if (2.0*abs(fret-fp) <= ftol*(abs(fret)+abs(fp)+EPS))
        return p;
    fp=fret;
    func.df(p,xi);
    Doub test=0.0;                     Test for convergence on zero gradient.
    Doub den=MAX(fp,1.0);
    for (Int j=0;j<n;j++) {
        Doub temp=abs(xi[j])*MAX(abs(p[j]),1.0)/den;
        if (temp > test) test=temp;
    }
    if (test < GTOL) return p;         The other possible return.
    dgg=gg=0.0;
    for (Int j=0;j<n;j++) {
        gg += g[j]*g[j];
        dgg += xi[j]*xi[j];
        dgg += (xi[j]+g[j])*xi[j];
    }
    if (gg == 0.0)                     Unlikely. If gradient is exactly zero, then
        return p;                     we are already done.
    Doub gam=dgg/gg;
    for (Int j=0;j<n;j++) {
        g[j] = -xi[j];
        xi[j]=h[j]=g[j]+gam*h[j];
    }
}
throw("Too many iterations in frprmn");
}
};

```

10.8.1 Note on Line Minimization Using Derivatives

Kindly reread §10.6. We here want to do the same thing, but using derivative information in performing the line minimization. Simply replace all occurrences of `Linemethod` in `Frprmn` with `Dlinemethod`. The routine `Dlinemethod` is exactly the same as `Linemethod` except that `Brent` is replaced by `Dbrent` and `F1dim` by `Df1dim`:

```

template <class T>
struct Dlinemethod {
    Base class for line-minimization algorithms using derivative information. Provides the line-
    minimization routine linmin.
    VecDoub p;
    VecDoub xi;
    T &func;
    Int n;
    Dlinemethod(T &funcc) : func(funcc) {}
    Constructor argument is the user-supplied function or functor to be minimized.
    Doub linmin()
    Line-minimization routine. Given an n-dimensional point p[p[0]..p[n-1]] and an n-dimensional
    direction xi[xi[0]..xi[n-1]], moves and resets p to where the function or functor func(p) takes on

```

`mins_ndim.h`

a minimum along the direction `xi` from `p`, and replaces `xi` by the actual vector displacement that `p` was moved. Also returns the value of `func` at the returned location `p`. All of this is actually accomplished by calling the routines `bracket` and `minimize` of `Dbrent`.

```
{
    Doub ax,xx,xmin;
    n=p.size();
    Df1dim<T> df1dim(p,xi,func);
    ax=0.0;                                Initial guess for brackets.
    xx=1.0;
    Dbrent dbrent;
    dbrent.bracket(ax,xx,df1dim);
    xmin=dbrent.minimize(df1dim);
    for (Int j=0;j<n;j++) {                Construct the vector results to return.
        xi[j] *= xmin;
        p[j] += xi[j];
    }
    return dbrent.fmin;
}
};
```

`mins_ndim.h`

```
template <class T>
struct Df1dim {
    Must accompany linmin in Dlinemethod.
    const VecDoub &p;
    const VecDoub &xi;
    Int n;
    T &funcd;
    VecDoub xt;
    VecDoub dft;
    Df1dim(VecDoub_I &pp, VecDoub_I &xii, T &funcdd) : p(pp),
        xi(xii), n(pp.size()), funcd(funcdd), xt(n), dft(n) {}
    Constructor takes as inputs an n-dimensional point p[0..n-1] and an n-dimensional direc-
    tion xi[0..n-1] from linmin, as well as the functor funcd.
    Doub operator()(const Doub x)
    Functor returning value of the given function along a one-dimensional line.
    {
        for (Int j=0;j<n;j++)
            xt[j]=p[j]+x*xi[j];
        return funcd(xt);
    }
    Doub df(const Doub x)
    Returns the derivative along the line.
    {
        Doub df1=0.0;
        funcd.df(xt,dft);
        for (Int j=0;j<n;j++)
            df1 += dft[j]*xi[j];
        return df1;
    }
};
```

Dbrent always evaluates the derivative at the same value as the function, so `xt` is unchanged.

CITED REFERENCES AND FURTHER READING:

- Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press), §2.3.[1]
 Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1.7 (by K.W. Brodlie).[2]
 Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §8.7.

10.9 Quasi-Newton or Variable Metric Methods in Multidimensions

The goal of *quasi-Newton* methods, which are also called *variable metric* methods, is not different from the goal of conjugate gradient methods: to accumulate information from successive line minimizations so that N such line minimizations lead to the exact minimum of a quadratic form in N dimensions. In that case, the method will also be quadratically convergent for more general smooth functions.

Both quasi-Newton and conjugate gradient methods require that you are able to compute your function's gradient, or first partial derivatives, at arbitrary points. The quasi-Newton approach differs from the conjugate gradient in the way that it stores and updates the information that is accumulated. Instead of requiring intermediate storage on the order of N , the number of dimensions, it requires a matrix of size $N \times N$. Generally, for any moderate N , this hardly matters.

On the other hand, there is not, as far as we know, any overwhelming advantage that the quasi-Newton methods hold over the conjugate gradient techniques, except perhaps a historical one. Developed somewhat earlier, and more widely propagated, the quasi-Newton methods have by now developed a wider constituency of satisfied users. Likewise, some fancier implementations of quasi-Newton methods (going beyond the scope of this book; see below) have been developed to a greater level of sophistication on issues like the minimization of roundoff error, handling of special conditions, and so on.

Quasi-Newton methods come in two main flavors. One is the *Davidon-Fletcher-Powell (DFP)* algorithm (sometimes referred to as simply *Fletcher-Powell*). The other goes by the name *Broyden-Fletcher-Goldfarb-Shanno (BFGS)*. The BFGS and DFP schemes differ only in details of their roundoff error, convergence tolerances, and similar “dirty” issues that are outside of our scope [1,2]. However, it has become generally recognized that, empirically, the BFGS scheme is superior in these details. We will implement BFGS in this section.

As before, we imagine that our arbitrary function $f(\mathbf{x})$ can be locally approximated by the quadratic form of equation (10.8.1). We don't, however, have any information about the values of the quadratic form's parameters $\mathbf{A}$ and $\mathbf{b}$, except insofar as we can glean such information from our function evaluations and line minimizations.

The basic idea of the quasi-Newton method is to build up, iteratively, a good approximation to the inverse Hessian matrix $\mathbf{A}^{-1}$, that is, to construct a sequence of matrices $\mathbf{H}_i$ with the property

$$\lim_{i \rightarrow \infty} \mathbf{H}_i = \mathbf{A}^{-1} \quad (10.9.1)$$

Even better if the limit is achieved after N iterations instead of ∞ .

The reason that these methods are called *quasi-Newton* can now be explained. Consider finding a minimum by using Newton's method to search for a zero of the gradient of the function. Near the current point $\mathbf{x}_i$, we have to second order

$$f(\mathbf{x}) = f(\mathbf{x}_i) + (\mathbf{x} - \mathbf{x}_i) \cdot \nabla f(\mathbf{x}_i) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_i) \cdot \mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) \quad (10.9.2)$$

so

$$\nabla f(\mathbf{x}) = \nabla f(\mathbf{x}_i) + \mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) \quad (10.9.3)$$

In Newton's method we set $\nabla f(\mathbf{x}) = 0$ to determine the next iteration point:

$$\mathbf{x} - \mathbf{x}_i = -\mathbf{A}^{-1} \cdot \nabla f(\mathbf{x}_i) \quad (10.9.4)$$

The left-hand side is the finite step we need to take to get to the exact minimum; the right-hand side is known once we have accumulated an accurate $\mathbf{H} \approx \mathbf{A}^{-1}$.

The “quasi” in quasi-Newton is because we don't use the actual Hessian matrix of f , but instead use our current approximation of it. This is often *better* than using the true Hessian. We can understand this paradoxical result by considering the *descent directions* of f at $\mathbf{x}_i$. These are the directions $\mathbf{p}$ along which f decreases: $\nabla f \cdot \mathbf{p} < 0$. For the Newton direction (10.9.4) to be a descent direction, we must have

$$\nabla f(\mathbf{x}_i) \cdot (\mathbf{x} - \mathbf{x}_i) = -(\mathbf{x} - \mathbf{x}_i) \cdot \mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) < 0 \quad (10.9.5)$$

which is true if $\mathbf{A}$ is positive-definite. In general, far from a minimum, we have no guarantee that the Hessian is positive-definite. Taking the actual Newton step with the real Hessian can move us to points where the function is *increasing* in value. The idea behind quasi-Newton methods is to start with a positive-definite, symmetric approximation to $\mathbf{A}$ (usually the unit matrix) and build up the approximating $\mathbf{H}_i$'s in such a way that the matrix $\mathbf{H}_i$ remains positive-definite and symmetric. Far from the minimum, this guarantees that we always move in a downhill direction. Close to the minimum, the updating formula approaches the true Hessian and we enjoy the quadratic convergence of Newton's method.

When we are not close enough to the minimum, taking the full Newton step $\mathbf{p}$ even with a positive-definite $\mathbf{A}$ need not decrease the function; we may move too far for the quadratic approximation to be valid. All we are guaranteed is that *initially* f decreases as we move in the Newton direction. Once again we can use the backtracking strategy described in §9.7 to choose a step along the *direction* of the Newton step $\mathbf{p}$, but not necessarily all the way.

We won't rigorously derive the DFP algorithm for taking $\mathbf{H}_i$ into $\mathbf{H}_{i+1}$; you can consult [3] for clear derivations. Following Brodlie (in [2]), we will give the following heuristic motivation of the procedure.

Subtracting equation (10.9.4) at $\mathbf{x}_{i+1}$ from that same equation at $\mathbf{x}_i$ gives

$$\mathbf{x}_{i+1} - \mathbf{x}_i = \mathbf{A}^{-1} \cdot (\nabla f_{i+1} - \nabla f_i) \quad (10.9.6)$$

where $\nabla f_j \equiv \nabla f(\mathbf{x}_j)$. Having made the step from $\mathbf{x}_i$ to $\mathbf{x}_{i+1}$, we might reasonably want to require that the new approximation $\mathbf{H}_{i+1}$ satisfy (10.9.6) as if it were actually $\mathbf{A}^{-1}$, that is,

$$\mathbf{x}_{i+1} - \mathbf{x}_i = \mathbf{H}_{i+1} \cdot (\nabla f_{i+1} - \nabla f_i) \quad (10.9.7)$$

We might also imagine that the updating formula should be of the form $\mathbf{H}_{i+1} = \mathbf{H}_i + \text{correction}$.

What “objects” are around out of which to construct a correction term? Most notable are the two vectors $\mathbf{x}_{i+1} - \mathbf{x}_i$ and $\nabla f_{i+1} - \nabla f_i$, and there is also $\mathbf{H}_i$. There are not infinitely many natural ways of making a matrix out of these objects, especially if (10.9.7) must hold! One such way, the *DFP updating formula*, is

$$\begin{aligned} \mathbf{H}_{i+1} = \mathbf{H}_i + & \frac{(\mathbf{x}_{i+1} - \mathbf{x}_i) \otimes (\mathbf{x}_{i+1} - \mathbf{x}_i)}{(\mathbf{x}_{i+1} - \mathbf{x}_i) \cdot (\nabla f_{i+1} - \nabla f_i)} \\ & - \frac{[\mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)] \otimes [\mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)]}{(\nabla f_{i+1} - \nabla f_i) \cdot \mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)} \end{aligned} \quad (10.9.8)$$

where $\otimes$ denotes the “outer” or “direct” product of two vectors, a matrix: The ij component of $\mathbf{u} \otimes \mathbf{v}$ is $u_i v_j$. (You might want to verify that 10.9.8 does indeed satisfy 10.9.7.)

The *BFGS updating formula* is exactly the same, but with one additional term,

$$\cdots + [(\nabla f_{i+1} - \nabla f_i) \cdot \mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)] \mathbf{u} \otimes \mathbf{u} \quad (10.9.9)$$

where $\mathbf{u}$ is defined as the vector

$$\mathbf{u} \equiv \frac{(\mathbf{x}_{i+1} - \mathbf{x}_i)}{(\mathbf{x}_{i+1} - \mathbf{x}_i) \cdot (\nabla f_{i+1} - \nabla f_i)} - \frac{\mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)}{(\nabla f_{i+1} - \nabla f_i) \cdot \mathbf{H}_i \cdot (\nabla f_{i+1} - \nabla f_i)} \quad (10.9.10)$$

(You might also verify that this satisfies 10.9.7.)

You will have to take on faith — or else consult [3] for details of — the “deep” result that equation (10.9.8), with or without (10.9.9), does in fact converge to $\mathbf{A}^{-1}$ in N steps, if f is a quadratic form.

Here now is the routine `dfpmin` that implements the quasi-Newton method and uses `lnsrch` from §9.7. As mentioned at the end of `newt` in §9.7, this algorithm can fail if your variables are badly scaled. You must provide a functor with the same format as the one for `frprmn` in §10.8 to calculate the function and its gradient.

```
template <class T>
void dfpmin(VecDoub_I0 &p, const Doub gtol, Int &iter, Doub &fret, T &funcd)
// Given a starting point p[0..n-1], the Broyden-Fletcher-Goldfarb-Shanno variant of Davidon-
// Fletcher-Powell minimization is performed on a function whose value and gradient are provided
// by a functor funcd (see text in §10.8). The convergence requirement on zeroing the gradient
// is input as gtol. Returned quantities are p[0..n-1] (the location of the minimum), iter (the
// number of iterations that were performed), and fret (the minimum value of the function). The
// routine lnsrch is called to perform approximate line minimizations.
{
    const Int ITMAX=200;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    const Doub TOLX=4*EPS,STPMX=100.0;
    // Here ITMAX is the maximum allowed number of iterations; EPS is the machine precision;
    // TOLX is the convergence criterion on x values; and STPMX is the scaled maximum step length
    // allowed in line searches.
    Bool check;
    Doub den,fac,fad,fae,fp,stpmax,sum=0.0,sumdg,sumxi,temp,test;
    Int n=p.size();
    VecDoub dg(n),g(n),hdg(n),pnew(n),xi(n);
    MatDoub hessin(n,n);
    fp=funcd(p);
    funcd.df(p,g);
    for (Int i=0;i<n;i++) {
        for (Int j=0;j<n;j++) hessin[i][j]=0.0;
        hessin[i][i]=1.0;
        xi[i] = -g[i];
        sum += p[i]*p[i];
    }
    stpmax=STPMX*MAX(sqrt(sum),Doub(n));
    for (Int its=0;its<ITMAX;its++) {
        iter=its;
        lnsrch(p,fp,g,xi,pnew,fret,stpmax,check,funcd);
        // The new function evaluation occurs in lnsrch; save the function value in fp for the
        // next line search. It is usually safe to ignore the value of check.
        fp=fret;
        for (Int i=0;i<n;i++) {
            xi[i]=pnew[i]-p[i];
        }
    }
}
```

quasinewton.h

Calculate starting function value and gradient,
and initialize the inverse Hessian to the unit matrix.

Initial line direction.

Main loop over the iterations.

Update the line direction,

```

    p[i]=pnew[i];
}
test=0.0;
for (Int i=0;i<n;i++) {
    temp=abs(xi[i])/MAX(abs(p[i]),1.0);
    if (temp > test) test=temp;
}
if (test < TOLX)
    return;
for (Int i=0;i<n;i++) dg[i]=g[i];
funcd.df(p,g);
test=0.0;
den=MAX(fret,1.0);
for (Int i=0;i<n;i++) {
    temp=abs(g[i])*MAX(abs(p[i]),1.0)/den;
    if (temp > test) test=temp;
}
if (test < gtol)
    return;
for (Int i=0;i<n;i++)
    dg[i]=g[i]-dg[i];
for (Int i=0;i<n;i++) {
    hdg[i]=0.0;
    for (Int j=0;j<n;j++) hdg[i] += hessin[i][j]*dg[j];
}
fac=fae=sumdg=sumxi=0.0;
for (Int i=0;i<n;i++) {
    fac += dg[i]*xi[i];
    fae += dg[i]*hdg[i];
    sumdg += SQR(dg[i]);
    sumxi += SQR(xi[i]);
}
if (fac > sqrt(EPS*sumdg*sumxi)) {
    fac=1.0/fac;
    fad=1.0/fae;
    The vector that makes BFGS different from DFP:
    for (Int i=0;i<n;i++) dg[i]=fac*xi[i]-fad*hdg[i];
    for (Int i=0;i<n;i++) {
        The BFGS updating formula:
        for (Int j=i;j<n;j++) {
            hessin[i][j] += fac*xi[i]*xi[j]
                -fad*hdg[i]*hdg[j]+fae*dg[i]*dg[j];
            hessin[j][i]=hessin[i][j];
        }
    }
}
for (Int i=0;i<n;i++) {
    xi[i]=0.0;
    for (Int j=0;j<n;j++) xi[i] -= hessin[i][j]*g[j];
}
}
throw("too many iterations in dfpmin");
}

```

and the current point.

Test for convergence on Δx .

Save the old gradient,
and get the new gradient.
Test for convergence on zero gradient.

Compute difference of gradients,
and difference times current matrix.

Calculate dot products for the denominators.

Skip update if fac not sufficiently positive.

The BFGS updating formula:

Now calculate the next direction to go,

and go back for another iteration.

Quasi-Newton methods like `dfpmin` work well with the approximate line minimization done by `lnsrch`. The routines `Powell` (§10.7) and `Frprmn` (§10.8), however, need more accurate line minimization, which is carried out by the routine `linmin` in `Linemethod` or `Dlinemethod`.

10.9.1 Quasi-Newton Methods Without Derivatives

In using Newton's method to find a zero of a function in multidimensions, we saw in §9.7 that one can use finite differences to calculate the partial derivatives